📍 Opprinnelig visjon

Du ønsket:

- Et lokalt AI-system med flere spesialiserte agenter som jobber sammen (et "råd").
- En agent som overvåker og forbedrer systemet (Petter Smart).
- Evne til å utvikle apper og nettsider autonomt.
- Sikkerhet og stabilitet.

🧱 Fase 1: Grunnmuren

1.1 Hardware og arkitektur

- **Threadripper** (100.103.79.103) – hovedmaskin med 35 GB VRAM, 62 GB RAM, 1.2 TB lagring
- ~~**Lenovo** (100.108.91.44)~~ – opprinnelig sandkasse, men ble utilgjengelig
- **Acer** (100.114.112.61) – kontrollsentral/overvåking

Utfordring: Lenovo ble borte – vi måtte konsolidere alt på Threadripper.

1.2 Installere 12 AI-modeller

Vi lastet ned alle nødvendige modeller i Ollama på Threadripper:

- DeepSeek-Coder-V2 16B (kode)
- Qwen2.5 14B/7B (strategi/forskning)
- Mistral 7B (backup)
- Llama 3.1/3.2 (allsidig/rask)
- CodeQwen 7B (tester)
- DeepSeek-R1 7B (debugger)

Resultat: 10 funksjonelle modeller (noen varianter er slått sammen).

1.3 Etablere tjenester på verten

- **Anita AI (AnythingLLM)** – port 3001
- **AI Agent Office** (3D-avatarer) – port 3002 (måtte startes med `--bind 0.0.0.0`)
- **Protokoll-server** (`enkel_server.py`) – port 8765 (måtte endres til å lytte på 0.0.0.0)
- **Brekkestranda nettside** – port 8080

🐳 Fase 2: Introduksjon av Kubernetes (K3s) og Consul

2.1 Hvorfor Kubernetes?

- **Selvhelbredende** – hvis en prosess krasjer, starter den automatisk
- **Tjenesteoppdagelse** – agenter finner hverandre via navn, ikke IP-adresser
- **Isolasjon** – hver agent kjører i sin egen container
- **Skalerbarhet** – enkelt å kjøre flere instanser

2.2 Oppsett av K3s via K3d

Vi installerte K3d (K3s i Docker) på Threadripper:

```powershell
k3d cluster create ai-studio --servers 1 --agents 0 --port "8080:80@loadbalancer"
--port "8443:443@loadbalancer"
```

◣◣

2.3 Installere Consul i Kubernetes

Consul ble installert med Helm og konfigurert for Windows-støtte (selv om nodene er Linux). ConnectInject ble aktivert, slik at alle pods med riktig annotasjon automatisk registreres i tjenestekatalogen.

Test: `test-nginx`-pod ble opprettet og registrert i Consul – bekreftet at auto-registrering fungerte.

🤖 Fase 3: Flytte agenter til Kubernetes

3.1 Petter Smart – den første agenten i Kubernetes

Utfordringer underveis:

1. **Hardkodede Windows-stier** – måtte endres til relative stier i Linux-container
2. **Tilgang til vertstjenester** – pod'er kan ikke nå `localhost` på verten, må bruke `host.docker.internal`
3. **ImagePullBackOff** – måtte sette `imagePullPolicy: Never` fordi vi importerer images lokalt
4. **Logging** – måtte endre `log_message` i `enkel_server.py` til stdout for å se logger i `kubectl logs`

Løsninger:

- Endret alle stier til `Path(__file__).parent / "logs"` etc.
- Satt `THREADRIPPER_IP = "host.docker.internal"` i Petter Smart
- La til `imagePullPolicy: Never` i deployment
- Omskrev logging til stdout

Status: Petter Smart kjører nå i Kubernetes, overvåker vertstjenester (Anita AI, AI Agent Office, Ollama) via `host.docker.internal` og protokoll-serveren internt.

3.2 Protokoll-server – flytting til Kubernetes

Utfordringer:

1. **Manglet** `sandkasse_protokoll.py` i Docker-image – fikset med COPY
2. **Logging til Windows-sti** – måtte endres til stdout
3. **Tilkobling til Ollama** – prøvde å nå `localhost`, måtte endres til `host.docker.internal`

Løsninger:

- Endret `log_message` til å bruke `print` (stdout)
- Endret `LocalOllamaClient` i `sandkasse_protokoll.py` til å bruke `host.docker.internal:11434`
- Bygget nytt image og importerte til K3d

Status: Protokoll-server kjører i Kubernetes, svarer på `http://protokoll:8765/status` og rapporterer `ollama_tilkoblet: true`.

3.3 Oppdatere Petter Smart til å bruke tjenestenavn

Etter at protokoll-serveren var i Kubernetes, endret vi Petter Smart til å bruke `http://protokoll:8765` i stedet for `host.docker.internal`. Dette gjør systemet mer Kubernetes-nativt og reduserer avhengighet av verts-IP.

📊 Nåværende systemarkitektur

Fysisk

```text
Threadripper (Windows 11)
├── Docker Desktop (med WSL2)
│   └── K3d cluster "ai-studio"
│       ├── Node: k3d-ai-studio-server-0 (Linux)
│       └── Kubernetes pods
├── Tjenester direkte på verten
│   ├── Anita AI (AnythingLLM) :3001
│   ├── AI Agent Office :3002
│   ├── Ollama API :11434
│   └── Brekkestranda nettside :8080
└── Acer (overvåking)
    └── enkel_monitor.py
```

Kubernetes

```text
Namespace: default
├── Deployment: pettersmart
│   └── Pod med 2 containers (pettersmart + consul-dataplane)
├── Deployment: protokoll
│   └── Pod med 2 containers (protokoll + consul-dataplane)
├── Service: protokoll (ClusterIP, port 8765)
└── Pod: test-nginx (test av Consul)
```

```
Namespace: consul
├── Deployment: consul-server
├── Deployment: consul-connect-injector
├── Service: consul-ui (NodePort :30500)
└── ...
```

◣◣

Tjenesteoppdagelse

- **Petter Smart** når vertstjenester via `host.docker.internal`
- **Petter Smart** når protokoll-server via `protokoll:8765`
- **Protokoll-server** når Ollama via `host.docker.internal:11434`
- Alle pods med `consul.hashicorp.com/connect-inject: "true"` registreres automatisk i Consul

✅ Fullført funksjonalitet

Hva	Hvordan	Status
12 AI-modeller	Ollama på verten	✅ 10 modeller (funksjonelt dekket)
Anita AI (prosjektleder)	AnythingLLM på verten	✅
AI Agent Office (avatarer)	HTTP-server på verten	✅
Petter Smart (meta-agent)	Kubernetes	✅ overvåker alt
Protokoll-server	Kubernetes	✅ tilgjengelig internt
Consul-tjenesteoppdagelse	Automatisk registrering	✅
Brekkestranda nettside	HTTP-server på verten	✅
Overvåking fra Acer	Python-skript	✅

📈 Veien videre – prioriterte neste steg

1. Forbedre Petter Smart med intelligente anbefalinger

- La Petter Smart analysere logger og gi forslag til forbedringer (f.eks. bytte modell, justere prompter).
- Integrere Gemini/DeepSeek API for avansert analyse.
- Lagre erfaringer i en database (ikke bare JSON-fil).

2. Gjøre protokoll-serverens Ollama-tilkobling konfigurerbar via miljøvariabel

- I stedet for hardkodet `host.docker.internal`, la den lese `OLLAMA_URL` fra miljøet.
- Da kan den enkelt bytte mellom vert og annen Ollama-instans.

3. Flytte Anita AI (AnythingLLM) til Kubernetes

Dette er mer komplekst fordi AnythingLLM krever:

- PostgreSQL-database
- Persistent lagring for dokumenter
- Web-grensesnitt (som må eksponeres)

Fordeler: Selvhelbredende, enklere backup, kan skaleres.

4. Oppgradere AI Agent Office

- Koble den til protokoll-serveren via API for å vise sanntidsstatus.
- Legg til WebSocket for levende oppdateringer når agenter jobber.

5. Bygge et samlet overvåkingsdashbord

- Samle logger fra Kubernetes og verten i Grafana.
- Vis CPU/GPU-bruk, modellbruk, feilrater.

6. Integrere Gemini/DeepSeek/Kimi API-er

- For bildegenerering (hotellbilder)
- For avansert kvalitetssikring av kode

- For "tenke utenfor boksen"-oppgaver

7. Containerisere AI Agent Office og Anita AI (langsiktig)

- Gjøre hele systemet Kubernetes-native
- Én maskin, ett orkestreringslag, full kontroll

🧰 Vedlikehold – daglig bruk

Starte alt (hvis maskinen restartes)

På verten (Threadripper):

```powershell
# AI Agent Office
cd C:\AI_System\ai-viz
python -m http.server 3002 --bind 0.0.0.0

# Brekkestranda nettside
cd C:\AI_System\produksjon\brekkestranda-fjordhotell
python -m http.server 8080

# Anita AI starter automatisk (tjeneste)
# Ollama starter automatisk (tjeneste)
```

◣◣

Kubernetes starter automatisk når Docker kjører. Sjekk at clusteret er oppe:

```powershell
k3d cluster list
kubectl get nodes
```

◣◣

Sjekke logger

```powershell
# Petter Smart
kubectl logs -l app=pettersmart -c pettersmart --tail=50
```

```
# Protokoll-server
kubectl logs -l app=protokoll -c protokoll --tail=50

# Consul UI
kubectl port-forward -n consul svc/consul-ui 8501:8500
# Åpne http://localhost:8501
```

◣◣

Oppdatere en agent (f.eks. Petter Smart)

```powershell
cd C:\AI_System\docker\pettersmart
docker build -t pettersmart:latest .
k3d image import pettersmart:latest -c ai-studio
kubectl delete deployment pettersmart
kubectl apply -f pettersmart-deployment.yaml
```

◣◣

📝 Oppsummering av endringer vi har gjort

I kode:

- `petter_smart.py`:
  - Endret til relative stier (Path(**file**).parent)
  - Byttet fra `localhost` til `host.docker.internal` for vertstjenester
  - Byttet fra `host.docker.internal:8765` til `protokoll:8765` for intern protokoll
  - La til støtte for psutil (valgfritt)
- `enkel_server.py`:
  - Endret `log_message` til å bruke stdout (print)
  - Fjernet forsøk på å skrive til Windows-sti
- `sandkasse_protokoll.py`:
  - Endret `LocalOllamaClient` fra `localhost:11434` til `host.docker.internal:11434`

I infrastruktur:

- Opprettet K3d-cluster
- Installert Consul med Helm
- Opprettet deployments og services for Petter Smart og protokoll
- Satt `imagePullPolicy: Never` for lokale images
- Testet med `test-nginx` for å verifisere Consul-registrering

I dokumentasjon:

- Laget fullstendige oversikter over tjenester, adresser og modeller
- Dokumentert kommandoer for vedlikehold

🎯 Konklusjon

Du har nå et **hybrid AI-studio** som kombinerer:

- Stabiliteten og selvhelbredelsen fra Kubernetes
- Kraften fra lokale LLM-er (10 modeller)
- Et overvåkingssystem (Petter Smart) som holder orden
- En protokoll-server for prosjektstyring
- Et visuelt grensesnitt (AI Agent Office) med 12 avatarer
- En fungerende nettside for Brekkestranda Fjordhotell

Alt kjører på **én maskin** (Threadripper), men med Kubernetes sin isolasjon og tjenesteoppdagelse. Systemet er klart for videre utvikling – enten det er å gjøre Petter Smart smartere, flytte flere komponenter til Kubernetes, eller integrere skybaserte API-er.

Gratulerer! Du har bygget noe virkelig imponerende. 🚀

Spørsmål? Ønsker du å begynne på et av de neste stegene?